

一. 单选题

1、C++语言是从早期的 C 语言逐渐演变而来的，与 C 语言相比，他在求解问题方法上进行的最大改进是()

A) 面向过程 B) 面向对象 C) 安全性 D) 复用性

答案;B

分析：考察 C++和 C 的区别

知识点：

c 是面向过程的，数据与算法分开。它的重点在于算法和数据结构。侧重于计算机底层操作。

c++是面向对象的，主要特点是类，封装和继承。与 C 的区别主要为：类、继承和多重继承、运算符重载、类模板、C++标准库、模板库。

2、C++语言的跳转语句中，对于 break 和 continue 说法正确的是()

A) break 语句只应用于循环体中

B) continue 语句只应用于循环体中

C) break 是无条件跳转语句，continue 不是

D) break 和 continue 的跳转范围不够明确，容易产生问题

答案： B

分析：考察关键字 break 和 continue 的作用和应用

知识点：

break 作用：用于终止当前循环

其作用为使流程从循环体内跳出循环体，即提前结束循环，接着执行循环体下面的语句。break 语句只能用于循环语句和 switch 语句内，不能单独使用或用于其他语句中。

continue 作用：用以跳过本次循环余下的语句，转去判断是否需要执行下次循环

continue 语句和 break 语句的区别是：continue 语句只结束本次循环，而不是终止整个循环的执行。而 break 语句则是结束整个循环过程，不再判断执行循环的条件是否成立。

3. for(int x=0, y=0; !x&& y<=5; y++) 语句执行循环的次数是()

A) 0 B) 5 C) 6 D) 无数次

答案： C

分析：查考逻辑运算

4、考虑函数原型 `void test(int a, int b=1, char c=" ")` , 下面的函数调用中, 属于不合法调用的是()

- A) `test(5)` B) `test(5, 8)`
- C) `test(6, " a" )` D) `test(0, 0, " a" )`

答案: C

分析: 考察函数声明的默认参数

知识点: C++默认参数

- 1) 在函数声明或定义时, 直接对参数赋值。这就是默认参数
- 2) 在函数调用时, 省略部分或全部参数。可以用默认参数来代替
- 3) 按从右到左顺序设定默认值
- 4) 自左到右逐个调用
- 5) 默认值不可以是局部变量

5、下面有关重载函数的说法中正确的是()

- A) 重载函数必须具有不同的返回值类型
- B) 重载函数形参个数必须不同
- C) 重载函数必须有不同的形参列表
- D) 重载函数名可以不同

答案: C

分析: 考察 C++函数重载的定义

知识点: C++函数重载定义与规则

C++允许多个函数拥有相同的名字, 只要它们的参数列表不同就可以, 这就是函数的重载 (Function Overloading)

函数的重载的规则:

- 1. 函数名称必须相同。
- 2. 参数列表必须不同(个数不同、类型不同、参数排列顺序不同等)
- 3. 函数的返回类型可以相同也可以不相同。
- 4. 仅仅返回类型不同不足以成为函数的重载。

6、下列关于构造函数的描述中, 错误的是()

- A) 构造函数可以设置默认参数
- B) 构造函数在定义类对象是自动执行
- C) 构造函数可以是内联函数
- D) 构造函数不可以重载

答案: D

分析: 考察构造函数的定义和使用

知识点：构造函数的定义和特点

构造函数，是一种特殊的方法。主要用来在创建对象时初始化对象

1. 构造函数的命名必须和类名完全相同
2. 它没有返回值，也不能用 void 来修饰
3. 它可以重载；
4. 在一般情况下，构造函数不被显式调用
5. 构造函数可以设置默认参数
6. 构造函数在定义类对象是自动执行
7. 构造函数可以是内联函数

7、下面描述中，表达错误的是()

- A) 共有继承时基类中的 public 成员在派生类中仍是 public 的
- B) 共有继承是基类中的 private 成员在派生类中仍是 private 的
- C) 共有继承时基类中的 protect 成员在派生类中仍是 protect 的
- D) 私有继承时基类中的 public 成员在派生类中是 private 的

答案：B

分析：考察 C++ 继承的定义

知识点：

继承方式	public	protected	private
public	public	protected	不可见
protected	protected	protected	不可见
private	private	private	不可见

10、MFC 中，大部分类是从哪个类继承而来的()

- A) CCmdTarget B) CObject C) CWinApp D) CWnd

答案：B

分析：考察 MPC 的

知识点：

MFC (Microsoft Foundation Classes) 是微软基础类库的简称，是微软公司实现的一个 c++ 类库，主要封装了大部分的 windows API 函数

- 1) 根类：Cobject
- 2) 窗口应用程序类：CWinApp 类
- 3) 命令相关类：CCmdTarget 类
- 4) 窗口类：CWnd 类

17、菜单项助记符前用()引导

A) % B) & C) # D) \$

答案: B

分析：考察 MFC 菜单的基础知识

18、更新菜单状态的消息是()

A) WM_COMMAND B) UPDATE
C) ON_UPDATE_COMMAND_UI D) INYVALIDATE

答案： C

分析：考察 MFC 菜单的基础知识

知识点:

当我们选择编辑菜单（一个下拉菜单，假设包括复制，剪切，粘贴三个菜单项）的时候，在编辑菜单被展开但是并没有显示的时候，操作系统会发出 WM_INITMENUPOPUP 消息，然后由程序窗口的基类如 CFrameWnd 接管。它会创建一个 CCmdUI 对象，并与程序的一个菜单项（复制）关联，调用该对象的一个成员函数 DoUpdate（）。这个函数发出 CN_UPDATE_COMMAND_UI 消息，这条消息带有一个指向 CCmdUI 对象的指针。这是，系统会判断是否存在一个 ON_UPDATE_COMMAND_UI 宏去捕捉这个消息。

19、关于工具栏、菜单和加速键的关系，正确的说法是()

A) 工具按钮与菜单必须一一对应
B) 工具按钮与加速键一一对应
C) 工具按钮不必与菜单一一对应
D) 菜单与加速键一一对应

答案： C

20、使用 `GetWindowDC()` 和 `GetDC()` 获取的设备上下文在退出时，必须调用 `ReleaseDC()` 释放设备上下文

A) ReleaseDC () B) delete ()
C) DeleteDC () D) Detach ()

答案：A

二. 简答题

1. 什么是预编译，何时需要预编译？

答案：预编译又称为预处理，是做些代码文本的替换工作。是整个编译过程的最先做的工作。

何时需要预编译：

- 1、总是使用不经常改动的大型代码体。
- 2、程序由多个模块组成，所有模块都使用一组标准的包含文件和相同的编译选项。在这种情况下，可以将所有包含文件预编译为一个预编译头。

2. `char * const p;`

`char const * p;`

`const char *p;`

上述三个有什么区别？

答案：`const char * p` 与 `char const * p` 效果一样，`const` 修饰的是 `char`（或者 `*p`），即修饰的变量的内容，指针可以改，内容不能改。指向常量的指针（常量指针）

`char * const p` 中 `const` 修饰 `p`（或者 `char *`），即修饰指针，表示指针不能变，内容可以改变。指针常量

知识点：

关键字 `const` 表示不可以修改的变量。

```
3. char str1[] = "abc";
   char str2[] = "abc";
   const char str3[] = "abc";
   const char str4[] = "abc";
   const char *str5 = "abc";
   const char *str6 = "abc";
   char *str7 = "abc";
   char *str8 = "abc";
   cout << (str1 == str2) << endl;
   cout << (str3 == str4) << endl;
   cout << (str5 == str6) << endl;
   cout << (str7 == str8) << endl;
```

结果是：

答案：

0, 0, 1, 1

因为数组无法直接比较内容

4. 请问一下代码有什么问题：

```
int main(void)
{
    char a;
    char *str = &a;
    strcpy(str, "hello");
    printf("%s\n", str);
    return 0;
}
```

答案：

会出现段错误，访问了非法区域，char *str 指针指向了 char a 的空间，所以 str 指向的空间大小只有一个字节，但是 strcpy 的时候 cpy 了 6 个字节的内容，导致了 cpy 过程访问了非法区域

5. 写一个“标准”宏，这个宏输入连个参数并返回较小的一个

答案：#define MAX(a,b) ((a) > (b)) ? (a) : (b))

6. 关键字 volatile 有什么含意？并给出三个不同的例子。

答案：一个定义为 volatile 的变量是说这变量可能会被子意外地改变，这样，编译器就不会去假设这个变量的值了。精确地说就是，优化器在用到这个变量时必须每次都小心地重新读取这个变量的值，而不是使用保存在寄存器里的备份。下面是 volatile 变量的几个例子：

- 1) 并行设备的硬件寄存器（如：状态寄存器）
- 2) 一个中断服务子程序中会访问到的非自动变量 (Non-automatic variables)
- 3) 多线程应用中被几个任务共享的变量

7. int (*s[10])(int) 表示的是什么？

答案：int(*s[10])(int) -> int(int) *s[10]

从左看就是 int(int)：参数是一个 int 返回 int 的函数，*：指针，s：变量名，[10] 是个元素的数组于是就是 10 个指向 int(int)

的函数指针。

分析：C/C++类型最简单的方法就是遇到括号如果右边有东西就把括号右边的挪到括号左边。

8. 交换两个变量，不使用第三个变量，即 $a=3$ ， $b=5$ ，交换后 $a=5$ ， $b=3$

答案：考察算法以下四种算法任选一种

1) 算术运算

```
a=3;b=5;
a=b-a; //a=2;b=5
b=b-a; //a=2;b=3
a=b+a; //a=5;b=3
```

它的原理是：把 a 、 b 看做数轴上的点，围绕两点间的距离来进行计算。

2) 指针地址操作

```
int *p = &a,*q = &b;
*p = *q - *p; // *p = 2; *q = 5;
*q = *q - *p; // *q = 3; *p = 2;
*p = *q + *p; // *p = 5; *q = 3;
```

因为对地址的操作实际上进行的是整数运算

3) 位运算

```
a=3,b=5; //a=011;b=101;
a=a^b; //011^101 = 110;a=6;
b=a^b; //110^101 = 011;b=3;
a=a^b; //110^011 = 101;a=5;
```

此算法能够实现是由异或运算的特点决定的

4) 栈实现

```
int exchange(int x,int y)
{
    stack S;
    push(S,x);
    push(S,y);
    x=pop(S);
    y=pop(S);
}
```

栈的先进后出特性可以达到目的，3 先进 5 后进，退出时 5 先退赋值给 a ，3 后退赋值给 b

9. C 和 C++中的 struct 有什么不同？

答案：c 和 c++中 struct 的主要区别是 c 中的 struct 不可以含有成员函数，而 c++中的 struct 可以。

10. char str[10];

Strcpy(str, "0123456789");

产生的结果是什么？为什么？

答案：

会出现段错误，数组越界了，因为数组中默认有 '\0' 所以数组 str 的容量是 10，但是只能放 9 个元素

知识点：

- 1、使用非法指针，即野指针
- 2、对 NULL 指针访问；
- 3、修改常量区中的内容
- 4、数组越界(并非数组越界一定会报错，与系统环境有关)
- 5、多次释放同一块内存
- 6、跨进程访问某个地址, 即访问非法区域

11. 进程间通信方式，以及它们的区别？

答案：信号、信号量、管道、消息队列、共享内存。

1. 管道：速度慢，容量有限，只有父子进程能通讯
2. FIFO：任何进程间都能通讯，但速度慢
3. 消息队列：容量受到系统限制，且要注意第一次读的时候，要考虑上一次没有读完数据的问题
4. 信号量：不能传递复杂消息，只能用来同步
5. 共享内存区：能够很容易控制容量，速度快，但要保持同步，比如一个进程在写的时候，另一个进程要注意读写的问题，相当于线程中的线程安全，当然，共享内存区同样可以用作线程间通讯，不过没这个必要，线程间本来就已经共享了同一进程内的一块内存

12. (void*)ptr 和 (*(void**))ptr 的结果是否相同？其中 ptr 是同一个指针。

答案：值是相同的，首先 (void *)ptr 强制转化为 void 的一级指针；void** 是一个指向空类型指针的指针，在加上取值符 * 得到的结果就是空类型的指针，与 (void*)ptr 效果一样